LinearSkills Academy

Class 14 — Advanced Functions

Beginner Friendly — Zero to Advance

Part 1 — Default Arguments

1. What is a Default Argument?

Definition: A value a parameter uses automatically if the caller doesn't provide one.

```
def greet(name="friend"):
    print("Hello,", name)

greet()          # Hello, friend
greet("Ali")      # Hello, Ali
```

Rule: default parameters must come after normal (non-default) parameters.

Part 2 — *args

2. What is *args?

Definition: Lets a function accept any number of values, collected into a tuple.

```
def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3))       # 6
print(total(5, 10, 15))     # 30
```

Part 3 — **kwargs

3. What is **kwargs?

Definition: Lets a function accept any number of named values, collected into a dictionary.

```
def show_info(**details):
    print(details)

show_info(name="Ali", age=20)
# {'name': 'Ali', 'age': 20}
```

4. Using Them Together

Order rule: normal parameters → *args → default parameters → **kwargs.

```
def demo(a, *args, **kwargs):
    print(a, args, kwargs)

demo(1, 2, 3, x=5)
# 1 (2, 3) {'x': 5}
```

Part 4 — Scope: Local vs Global

5. Local Scope

Definition: A variable made inside a function. It only exists inside that function.

```
def my_function():
    x = 10
    print(x)

my_function()
print(x)   # Error! x does not exist here
```

6. Global Scope

Definition: A variable made outside any function. It can be read anywhere.

```
x = 5

def show():
    print(x)   # can read global x

show()   # 5
```

7. The Local-Copy Trap

Changing a variable inside a function normally creates a new local one, and the global stays the same.

```
x = 5

def change():
    x = 10   # this is a NEW local x
    print(x)

change()      # 10
print(x)      # 5 (unchanged)
```

8. The global Keyword

Definition: Tells Python you want to change the real global variable, not make a local copy.

```
x = 5

def change():
    global x
    x = 10

change()
print(x)   # 10 (actually changed)
```

Part 5 — Python vs C++

Feature	Python	C++
Local variable	Exists inside function only	Exists inside { } block only
Global variable	Needs global keyword to change	Changeable directly

Variable-length args	*args, **kwargs	Templates / overloading (harder)
Default arguments	def f(x=5):	void f(int x = 5)

Part 6 — Small Project

Expense Tracker

A tiny project that uses everything from this class: *args, **kwargs, a default argument, and the global keyword.

```
total_expense = 0

def add_expense(*amounts, category="General", **details):
    global total_expense
    expense_sum = sum(amounts)
    total_expense += expense_sum

    print("Category:", category)
    print("Amounts:", amounts)
    print("Extra Info:", details)
    print("Added:", expense_sum)
    print("-" * 20)

add_expense(200, 150, category="Food", place="Karachi")
add_expense(500, category="Rent")
add_expense(80, 20, 10)

print("Total Expense:", total_expense)
```